Graph Theory Coding Task Report

Algorithmic Implementation & Complexity Analysis

Team Identity

Farras Al Ghifari – 5025251006
Nisrina Zahralila – 5025251013
Safiya Nadira Az-zahra – 5025251268
Naura Rizky Ameira – 5025251001

Overview

This document describes the algorithm idea, the implementation and the computational complexity of the three required simulators: the **Travelling Salesman Problem (SPPG Meal Delivery)**, the **Chinese Postman Problem (DLH Dump Truck)**, and **The Knight's Tour**.

Task 1: Travelling Salesman Problem (SPPG Simulator)

1. Algorithm Idea

The SPPG (Satuan Pelayanan Pemenuhan Gizi) simulator finds an efficient route that starts at an SPPG chosen on the map, visits every school within a user-selected radius x km, and returns to the SPPG.

1. **Network extraction:** The Surabaya road network and the school locations are downloaded from OpenStreetMap with OSMnx. The roads are stored as a directed graph (DiGraph), so one-way streets are respected, and each edge gets a travel speed from its maxspeed tag or its road class.
2. **Distance matrix:** Schools whose straight-line distance to the SPPG is at most x km are selected and snapped to their nearest road node. SciPy's directed Dijkstra then computes the shortest road distance between the SPPG and every selected school, giving a distance matrix d.
3. **Asymmetric TSP solver:** Because of one-way streets, $d(u,v) \neq d(v,u)$. A tour is first built with the **Nearest Neighbor** heuristic and improved by **Or-Opt local search**, which moves segments of 1–3 stops to a better position without reversing them (so it stays valid for asymmetric costs). An **exact** solver is then run as a second attempt: an integer linear program (MILP) with assignment constraints and subtour-elimination cuts that are added lazily. The better of the two tours is kept, and the interface reports whether the result is proven optimal.
4. **Route expansion:** Each leg of the tour is expanded into the real road path found by Dijkstra, so the route follows actual streets and may revisit road segments.

2. Implementation Details

- **Language & libraries:** Python, NetworkX, OSMnx, NumPy, SciPy, Flask and Leaflet.js (interactive map).
- **Key functions:** run_tsp() selects the schools in the radius, builds the distance matrix and expands the final route; solve_tsp() runs the heuristic and exact solvers; tsp_nearest_neighbor(), tsp_or_opt() and tsp_exact() implement the three solving steps.
- **Caching:** the road network is cached as surabaya_road_network.graphml and the schools as surabaya_schools.json, so OpenStreetMap is only queried once.
- **Output:** route overlaid on the map, school visiting order, total distance, estimated travel time and the TSP solver execution time.

3. Computational Complexity

Let V and E be the vertices and edges of the road graph, and let N be the number of stops (schools) inside the radius.

- **Distance matrix:** $O(N \cdot (V \log V + E))$, one directed Dijkstra search from the SPPG and from each school node.
- **Nearest Neighbor:** $O(N^2)$.
- **Or-Opt:** one pass tries about $3N$ segments and N insertion positions, each evaluated in $O(1)$ from the matrix, so one pass costs $O(N^2)$. Passes repeat while the tour improves. In our tests it ran in about 1 ms for 30 stops and about 5 ms for 50 stops.
- **Exact MILP:** TSP is NP-hard, so the worst case is exponential in N. The solver is therefore capped by a 30-second time limit, after which the best tour found so far is used.

Task 2: Chinese Postman Problem (DLH Dump Truck Simulator)

1. Algorithm Idea

The DLH simulator finds a closed route from a truck base chosen on the map that collects every trunk, primary, secondary and tertiary road (including their _link roads) within a user-selected radius x km. Other roads may be used only to connect or detour. Strictly, requiring only some roads makes this a **directed Rural Postman** problem, and we solve it with the Chinese Postman approach.

1. **Graph filtering:** The road network is restricted to its largest strongly connected component (SCC), so every node can reach every other node. The required directed edges are the trunk–tertiary edges whose midpoint lies within x km of the base. Required edges that are not connected to the base's component are ignored, as allowed by the task.
2. **Heuristic route:** The required edges are placed in a multigraph. Separate pieces are linked to the base's piece by shortest paths. Then, for each node, we compare incoming and outgoing degrees, $\Delta(v)$ = in-deg(v) − out-deg(v). Nodes with surplus incoming edges and nodes with surplus outgoing edges are matched by a **min-cost flow** over shortest-path distances, and the matched shortest paths are added as repeated edges until every node is balanced.
3. **Exact route:** An integer linear program minimizes the total length of the extra traversals subject to the balance constraints, with connectivity cuts added lazily. If it finds a route that is at least as short as the heuristic one, that route is used, and the result is marked optimal only when the solver proves it.
4. **Eulerian circuit:** A balanced, connected directed multigraph has an Eulerian circuit. It is found with NetworkX's eulerian_circuit (a Hierholzer-style algorithm) starting from the base, giving the full closed traversal order.

2. Implementation Details

- **Language & libraries:** Python, NetworkX, OSMnx, NumPy, SciPy, Flask and Leaflet.js.
- **Key functions:** select_required() filters the radius and the required edges; solve_cpp() runs the heuristic (_heuristic_multigraph()) and the exact solver (_exact_multigraph()) and builds the Eulerian circuit; build_segments() produces the street traversal order.
- **Caching:** the road network is cached as surabaya_road_network.graphml to avoid downloading it again.
- **Visualization:** the map shows collection passes and revisit or connector passes with different line styles (dashed lines for non-collection travel), together with the street traversal order, total distance, estimated travel time and the CPP solver execution time.

3. Computational Complexity

Let V and E be the vertices and edges of the road graph, S and T the numbers of nodes with surplus incoming and surplus outgoing edges, and E' the number of edges in the balanced multigraph.

- **SCC extraction:** $O(V + E)$ (Tarjan/Kosaraju).
- **Shortest paths for balancing:** one Dijkstra search from each surplus node, $O(S \cdot (E + V \log V))$.
- **Min-cost flow:** solved by NetworkX's network simplex on a bipartite graph with $S + T$ nodes and at most $S \cdot T$ arcs. It has no good polynomial bound, but this graph is small compared with the road network, so it is fast in practice.
- **Exact ILP:** exponential in the worst case, so it is capped by a 30-second time limit and the heuristic route is kept if it is not improved.
- **Eulerian circuit:** $O(E')$.

Task 3: The Knight's Tour

1. Algorithm Idea

The task is to move a knight on a $w \times h$ board, starting from a given square, so that it visits every square exactly once.

- **Warnsdorff's heuristic:** From the current square, the knight tries first the move that leads to the square with the **fewest onward moves** (ties are broken by the neighbours' own onward moves). This visits hard-to-reach squares before they become isolated, so the search rarely needs to backtrack.
- **Dead-end pruning:** If any reachable unvisited square has no onward move while other squares remain, that branch is abandoned immediately.
- **Backtracking DFS with an explicit stack:** If a dead end is still reached, the search backs up and tries the next candidate move. An explicit stack is used instead of recursion, so boards of 100 × 100 never hit Python's recursion limit.
- **Parity rule:** A knight alternates square colours. On a board with an odd number of squares, a tour must therefore start on a square with $(x + y)$ even, so other starts are rejected immediately.

2. Implementation Details

- **Libraries:** only the Python standard library.
- **Console mode:** run `python Chessknight.py` with no flags. It reads `w h` and then `x y` from standard input and prints one `x,y` per line in visiting order, or `path not found`.
- **Visual mode:** run `python Chessknight.py --visual` to start a local web server (`http.server`) with an HTML5 Canvas board. It animates the tour with play, step and speed controls, move numbers and path lines, and the same output text as the console version.
- **No-solution handling:** when no tour exists (for example 3 × 3, 4 × 4, or an odd-parity start on an odd board), the program prints `path not found`. In visual mode the longest partial path is shown. A 20-second time limit stops searches whose impossibility would take too long to prove.

3. Computational Complexity

- **Naive backtracking:** $O(8^{w \cdot h})$ in the worst case.
- **With Warnsdorff's heuristic:** when it succeeds without backtracking, each of the $w \cdot h$ steps checks at most 8 moves and a constant number of neighbour counts, so the time is about $O(w \cdot h)$. In our tests boards up to 100 × 100 were solved in under a second.
- **Worst case:** when backtracking is needed the running time can be exponential. The parity check handles the common impossible case instantly, and the 20-second limit bounds the rest.

AI Disclosure

TSP python code: https://chatgpt.com/share/6ad74eb8-8a40-83ec-b165-de0c804ff01d

CPP python code: https://claude.ai/share/2962d8b6-5f8b-4405-b342-3921ef373a97

Chessknight code: https://share.gemini.google/e3fvrm8uoc6r

Python code optimizing: https://share.gemini.google/MCybj4FQIBfe

Checking everything:

- https://share.gemini.google/GcVGPpqR4ChO
- https://claude.ai/share/d11a7a95-0bc4-4044-beea-4110c42443bd